API REFERENCE

PocketDex API

A fictional Pokédex-style API used as the privacy-safe reference document for this OpenAPI-to-Typst template. Every species, trainer, region and item below is invented for demonstration purposes.

1.0.0

SERVER

`https://api.pocketdex.example`

CONTACT

PocketDex Demo Team · hello@example.com

Generated 2026-10-21 · OpenAPI 3.1.0 · openapispec-typst 1.0.0

OPENAPI 3.1.0

14 ENDPOINTS · 47 SCHEMAS

Contents

PocketDex API · version 1.0.0

Overview ····· 1
Authentication & Authorization ····· 2
Request Conventions ····· 3
Response & Error Conventions ····· 4
Endpoint Summary ····· 7
Trainers ····· 8
  Trainer Login ····· 8
  Register Trainer ····· 9
  Update Trainer Party ····· 10
Species ····· 12
  List Species ····· 12
  Get Species ····· 13
  Get Species Stats (v1) ····· 14
  Get Species Stats (v2) ····· 15
  Legacy Species Search ····· 15
Encounters ····· 16
  Record Encounter ····· 16
  Get Encounter Order ····· 17
Regions ····· 18
  List Region Habitats ····· 18
Items ····· 19
  List Items ····· 19
  Import Item Catalogue ····· 20
Breeding ····· 21
  List Breeding Pairs ····· 21
Data Models ····· 22
Enumerations ····· 33
Specification Notes ····· 34

Overview

A fictional Pokédex-style API used as the privacy-safe reference document for this OpenAPI-to-Typst template. Every species, trainer, region and item below is invented for demonstration purposes.

14	47	6	1
ENDPOINTS	SCHEMAS	TAGS	BASE URLS

Intended audience

This reference is written for developers, QA engineers, architects and integration partners who build against the API.

Base URLs

Every endpoint path in this document is relative to one of the base URLs below.

`https://api.pocketdex.example`
Demo server (not a real service)

API groups

Trainers 3 →
Trainer profiles, registration and party management.

Species 5 →
Species discovery, detail pages and battle statistics.

Encounters 2 →
Record wild encounters and follow asynchronous orders.

Regions 1 →
Regions, habitats and the routes inside them.

Items 2 →
Item catalogue browsing and catalogue imports.

Breeding 1 →

Document details

VERSION	OPENAPI	GENERATED	PUBLISHER
1.0.0	3.1.0	2026-10-21	PocketDex Demo Team

Authentication & Authorization

Authentication is described once here. Endpoint sections only call out credentials that differ from the default.

DEFAULT REQUIREMENT

`TrainerToken` Applies to 13 of 14 endpoints.

Security schemes

TrainerToken `http / bearer`

Bearer token returned by the login endpoint.

HEADER FORMAT

```
Authorization: Bearer <token>
```

BasicAuth `http / basic`

Trainer credentials used only to obtain a bearer token.

HEADER FORMAT

```
Authorization: Basic <base64(clientId:clientSecret)>
```

No OAuth 2.0 scopes or roles are declared in this specification.

Endpoints with different authentication

Endpoint	Method	Security
Trainer Login	POST	BasicAuth

Request Conventions

Content types

`application/json` `multipart/form-data`

JSON is the default representation for requests and responses; multipart/form-data is used for file uploads.

Pagination

Where pagination is supported, endpoints accept the parameters below and responses include pagination metadata.

Name	Type	Required	Description
`pageNo`	`integer` default: 1 · min: 1	—	Page number, starting at 1. · Example: 1
`pageSize`	`integer` default: 20 · min: 1 · max: 100	—	Number of items per page. · Example: 20

Pagination metadata is defined by `Pagination`.

Sorting and filtering

Where supported, sorting selects the field and direction; filter applies field-level expressions.

Name	Type	Required	Description
`sortBy`	`string` enum: `name` | `rarity` | `speciesId` default: name	—	Field used for sorting.
`sortOrder`	↗ `SortOrder`	—	Sort direction.

Date and time

Date-time fields use ISO 8601 (RFC 3339) strings.

`2026-12-31T23:59:59Z`

Response & Error Conventions

Common conventions apply to every endpoint unless stated otherwise. Endpoint sections list endpoint-specific responses and link here for the shared failure responses; error payloads are shown once.

Error model

Error payloads are not uniform. Most endpoints return the standard object envelope below; the login endpoint returns a legacy array of error items.

`NewApiErrorResponse`

`object` 3 properties · 2 required

Standard error envelope used by most endpoints.

Field	Type	Req.	Constraints	Description
`error`	`string`	Yes		Stable machine-readable error code. Example: `pokedex.bad_request`
`message`	`string`	Yes		Human-readable summary of the failure. Example: `The request could not be understood.`
`details`	`array<` ↗ `ErrorDetail>`	—		Optional list of field-level details.

Status codes

Status codes used across the API; entries such as 4XX and 5XX represent status-class ranges.

Status	Meaning
200	OK
201	Created
202	Accepted
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
422	Unprocessable Content
429	Too Many Requests
500	Internal Server Error
4XX	Any 4xx client error
5XX	Any 5xx server error

Representative error examples

Structurally similar examples are shown once, taken verbatim from the specification.

Bad Request (400) `badRequest`

```
{
  "error": "pokedex.bad_request",
  "message": "The request could not be understood.",
  "details": [
    {
      "userMessage": "Check the request body and try again."
    }
  ]
}
```

Unauthorized (401) `unauthorized`

```
{
  "error": "pokedex.unauthorized",
  "message": "A valid trainer token is required.",
  "details": [
    {
      "userMessage": "Sign in again and retry the request."
    }
  ]
}
```

Forbidden (403) `forbidden`

```
{
  "error": "pokedex.forbidden",
  "message": "This trainer cannot access that record."
}
```

Not Found (404) `resourceNotFound`

```
{
  "error": "pokedex.not_found",
  "message": "The requested record does not exist.",
  "details": [
    {
      "userMessage": "Check the identifier and try again."
    }
  ]
}
```

Conflict (409) `conflict`

```
{
  "error": "pokedex.conflict",
  "message": "A record with that name already exists."
}
```

Invalid Input (422) `invalidInput`

```
{
  "error": "pokedex.invalid_input",
  "message": "One or more fields are invalid.",
  "details": [
    {
      "userMessage": "favoriteType must be a known type."
    }
  ]
}
```

Too Many Requests (429) `tooManyRequests`

```
{
  "error": "pokedex.rate_limited",
  "message": "Too many requests were sent in a short period."
}
```

Internal Server Error (500) `serverError`

```
{
  "error": "pokedex.internal_error",
  "message": "An unexpected error occurred."
}
```

Additional error payloads

ErrorDetail

`object` 1 properties · 1 required

A single human-readable error detail.

Field	Type	Req.	Constraints	Description
userMessage	string	Yes		Message suitable for display to a trainer. Example: `Check the identifier and try again.`

LegacyAuthError

`array<LegacyAuthErrorItem>`

Legacy array-shaped authentication error.

Array of ↗ LegacyAuthErrorItem

LegacyAuthErrorItem

`object` 2 properties · 2 required

Legacy authentication error item.

Field	Type	Req.	Constraints	Description
error	string	Yes		Legacy error code. Example: `pokedex.auth.invalid_credentials`
message	string	Yes		Legacy error message. Example: `The trainer name or password is incorrect.`

Success response envelopes

These response shapes are reused by multiple endpoints; endpoint sections link to the relevant entry here.

Accepted order response

`success envelope` AcceptedOrder

Schema: ↗ AcceptedOrder

Encounter order queued `orderAccepted`

```
{
  "orderId": "ord_demo_0001",
  "status": "QUEUED",
  "estimatedAt": "2026-04-01T10:00:05Z"
}
```

Endpoint Summary

Every endpoint in this specification. The detailed reference follows, grouped by API tag.

Method	Path	Purpose	Tag
POST	/api/v1/auth/login	Trainer Login ↗ non-default authentication	Trainers
POST	/api/v1/trainers	Register Trainer ↗	Trainers
PATCH	/api/v1/trainers/{trainerId}/party	Update Trainer Party ↗	Trainers
GET	/api/v1/species	List Species ↗	Species
GET	/api/v1/species/{speciesId}	Get Species ↗	Species
GET	/api/v1/species/{speciesId}/stats	Get Species Stats ↗	Species
GET	/api/v2/species/{speciesId}/stats	Get Species Stats ↗	Species
GET	/api/v1/species/legacy-search	Legacy Species Search ↗ DEPRECATED	Species
POST	/api/v1/encounters	Record Encounter ↗	Encounters
GET	/api/v1/encounters/{orderId}	Get Encounter Order ↗	Encounters
GET	/api/v1/regions/{regionId}/habitats	List Region Habitats ↗	Regions
GET	/api/v1/items	List Items ↗	Items
POST	/api/v1/items/import	Import Item Catalogue ↗	Items
GET	/api/v1/breeding/pairs	List Breeding Pairs ↗	Breeding

Trainers

Trainer profiles, registration and party management. 3 endpoints

Trainer Login

POST /api/v1/auth/login

Exchange trainer credentials for a bearer token. Use the returned token in the Authorization header for every other endpoint. The demo service accepts the credentials shown in the example.

Authentication: BasicAuth (non-default)

Request body Required

application/json Full model: TrainerLogin

Field	Type	Required	Description
username	string	Yes	Trainer username. · Example: mira.vale
password	string write-only	Yes	Trainer password. · Example: demo-password

Demo trainer login demoLogin

```
{
  "username": "mira.vale",
  "password": "demo-password"
}
```

Responses Common errors ↗

Status	Description	Schema
200	Successful login	↗ TrainerTokenResponse
401	Invalid trainer credentials	↗ LegacyAuthError

200 Response examples

Bearer token issued loginSuccess

```
{
  "token": "pkdx_demo_7f4c2a",
  "expiresAt": "2026-12-31T23:59:59Z",
  "trainer": {
    "trainerId": "trn_mira",
    "displayName": "Mira Vale",
    "region": "Lumen Vale"
  }
}
```

401 Response examples

Legacy authentication error legacyUnauthorized

```
[
  {
    "error": "pokedex.auth.invalid_credentials",
    "message": "The trainer name or password is incorrect."
  }
]
```

Register Trainer

`POST` `/api/v1/trainers`

Create a trainer profile. Registration is synchronous. Party management is available immediately after the profile is created.

Authentication: `TrainerToken` (default)

Request body Required

`application/json` Full model: `TrainerRegistration`

Field	Type	Required	Description
`displayName`	`string` min length: 2 · max length: 40	Yes	Public trainer name. · Example: `Mira Vale`
`homeRegion`	`string`	Yes	Name of the trainer's home region. · Example: `Lumen Vale`
`favoriteType`	↗ `PokemonType`	Yes	
`starterSpeciesId`	`string`	—	Optional starter species identifier. · Example: `spc_leafling`

Register a collector `mira`

```
{
  "displayName": "Mira Vale",
  "homeRegion": "Lumen Vale",
  "favoriteType": "GRASS",
  "starterSpeciesId": "spc_leafling"
}
```

Register with the minimum fields `minimal`

```
{
  "displayName": "New Trainer",
  "homeRegion": "Lumen Vale",
  "favoriteType": "NORMAL"
}
```

Showing 2 of 4 examples from the specification.

Responses Common errors ↗

Status	Description	Schema
`201`	Trainer profile created	↗ `TrainerCreatedResponse`

`201` Response examples

New trainer profile `created`

```
{
  "trainerId": "trn_mira",
  "displayName": "Mira Vale",
  "homeRegion": "Lumen Vale",
  "favoriteType": "GRASS",
  "createdAt": "2026-02-14T09:30:00Z"
}
```

Update Trainer Party

PATCH /api/v1/trainers/{trainerId}/party

Replace the active party for a trainer. The party may contain at most six members. The request is idempotent for the same trainer and payload.
Authentication: TrainerToken (default)

Path parameters

Name	Type	Required	Description
trainerId	string pattern: ^trn_[a-z0-9_]+$	Yes	Stable trainer identifier. · Example: trn_mira

Request body Required

application/json Full model: TrainerPartyUpdate

Field	Type	Required	Description
party	array<↗ PartyMember> max items: 6	Yes	Replacement party members.
favoriteType	↗ PokemonType	—	
notes	string max length: 200	—	Optional trainer notes. · Example: Keep the party balanced for cave routes.

Balanced party balanced

```
{
  "party": [
    {
      "speciesId": "spc_emberling",
      "level": 24,
      "role": "ATTACKER"
    },
    {
      "speciesId": "spc_aquafin",
      "level": 22,
      "role": "SUPPORT"
    }
  ],
  "favoriteType": "FIRE"
}
```

Defensive party defensive

```
{
  "party": [
    {
      "speciesId": "spc_rocklet",
      "level": 30,
      "role": "TANK"
    },
    {
      "speciesId": "spc_leafling",
      "level": 28,
      "role": "SUPPORT"
    }
  ],
  "favoriteType": "ROCK"
}
```

Responses Common errors ↗

Status	Description	Schema
200	Trainer party replaced	↗ TrainerPartyResponse

200 Response examples

Updated party `updatedParty`


```
{
  "trainerId": "trn_mira",
  "capacity": 6,
  "party": [
    {
      "speciesId": "spc_emberling",
      "level": 24,
      "role": "ATTACKER"
    }
  ]
}
```

Species

Species discovery, detail pages and battle statistics. 5 endpoints

List Species

GET /api/v1/species

Browse the species catalogue with pagination, sorting and optional habitat and type filters.
Authentication: TrainerToken (default)

Query parameters

Name	Type	Required	Description
habitat	string	—	Filter by habitat name. · Example: Sunlit Meadow
pageNo	integer default: 1 · min: 1	—	Page number, starting at 1. · Example: 1
pageSize	integer default: 20 · min: 1 · max: 100	—	Number of items per page. · Example: 20
sortBy	string enum: name | rarity | speciesId default: name	—	Field used for sorting.
sortOrder	↗ SortOrder	—	Sort direction.
type	↗ PokemonType	—	Filter by primary species type.

Responses Common errors ↗

Status	Description	Schema
200	A page of species	↗ SpeciesPage

200 Response examples

First page of species `speciesPage`


```
{
  "items": [
    {
      "speciesId": "spc_emberling",
      "name": "Emberling",
      "types": [
        "FIRE"
      ],
      "habitat": "Ember Cavern",
      "rarity": "UNCOMMON"
    },
    {
      "speciesId": "spc_aquafin",
      "name": "Aquafin",
      "types": [
        "WATER"
      ],
      "habitat": "Mistwood",
      "rarity": "COMMON"
    }
  ],
  "pagination": {
    "pageNo": 1,
    "pageSize": 20,
    "total": 42
  }
}
```


Get Species

GET `/api/v1/species/{speciesId}`

Return the complete species record, including abilities, base stats and evolution information.
Authentication: `TrainerToken` (default)

Path parameters

Name	Type	Required	Description
`speciesId`	`string`	Yes	Stable species identifier. · Example: `spc_emberling`

Responses

Common errors ↗

Status	Description	Schema
200	Species detail	↗ `SpeciesResponse`

200 Response examples

Emberling detail `emberling`


```
{
  "data": {
    "speciesId": "spc_emberling",
    "name": "Emberling",
    "types": [
      "FIRE"
    ],
    "habitat": "Ember Cavern",
    "rarity": "UNCOMMON",
    "description": "A small fox-like species that stores warmth in its tail.",
    "heightM": 0.6,
    "weightKg": 7.4,
    "abilities": [
      "Kindle",
      "Warm Coat"
    ],
    "stats": {
      "hp": 45,
      "attack": 60,
      "defense": 40,
      "speed": 65,
      "special": 55,
      "total": 265
    }
  }
}
```


Get Species Stats (v1)

GET /api/v1/species/{speciesId}/stats

Return the base battle statistics for a species.
Authentication: TrainerToken (default)

Path parameters

Name	Type	Required	Description
speciesId	string	Yes	Stable species identifier. · Example: spc_emberling

Responses Common errors ↗

Status	Description	Schema
200	Species statistics	↗ SpeciesStatsResponse

200 Response examples

Emberling base stats `stats`


```
{
  "hp": 45,
  "attack": 60,
  "defense": 40,
  "speed": 65,
  "special": 55,
  "total": 265
}
```

Get Species Stats (v2)

`GET` `/api/v2/species/{speciesId}/stats`

Return base statistics plus the latest balance revision details.
Authentication: `TrainerToken` (default)

Path parameters

Name	Type	Required	Description
`speciesId`	`string`	Yes	Stable species identifier. · Example: `spc_emberling`

Responses Common errors ↗

Status	Description	Schema
200	Species statistics with revision metadata	↗ `V2SpeciesStatsResponse`

200 Response examples

Emberling stats with balance revision `statsV2`

```
{
  "hp": 45,
  "attack": 60,
  "defense": 40,
  "speed": 65,
  "special": 55,
  "total": 265,
  "balanceRevision": "r7",
  "updatedAt": "2026-03-01T00:00:00Z"
}
```

Legacy Species Search

`GET` `/api/v1/species/legacy-search` DEPRECATED

Search species by name. This endpoint is retained for older clients.
Authentication: `TrainerToken` (default)

Query parameters

Name	Type	Required	Description
`q`	`string` min length: 1	Yes	Partial species name. · Example: `ember`

Responses Common errors ↗

Status	Description	Schema
200	Matching species	↗ `SpeciesSummaryList`

Encounters

Record wild encounters and follow asynchronous orders. 2 endpoints

Record Encounter

POST /api/v1/encounters

Queue a wild encounter report. The service processes encounters asynchronously and returns an order identifier that can be used to poll for progress.

Authentication: TrainerToken (default)

Request body Required

application/json Request variants: 3 alternatives

Variant	Required fields	When to use
SingleEncounterRequest	speciesId · level · location	Requires speciesId, level, location
PartyEncounterRequest	members · location	Submit a collection in members
RouteEncounterRequest	routeId · count · location	Submit an amount in count

Each variant is defined in full under Data Models; representative examples follow.

Single wild encounter single

```
{
  "speciesId": "spc_emberling",
  "level": 12,
  "location": "Ember Cavern"
}
```

Party encounter party

```
{
  "members": [
    {
      "speciesId": "spc_aquafin",
      "level": 11
    },
    {
      "speciesId": "spc_leafling",
      "level": 10
    }
  ],
  "location": "Mistwood"
}
```

Route sweep route

```
{
  "routeId": "rte_west_meadow",
  "count": 5,
  "location": "Sunlit Meadow"
}
```

Responses Common errors ↗

Status	Description	Schema
202	Accepted order response asynchronous envelope ↗	See envelope ↗

202 Accepted order response

Encounter order queued `orderAccepted`


```
{
  "orderId": "ord_demo_0001",
  "status": "QUEUED",
  "estimatedAt": "2026-04-01T10:00:05Z"
}
```


Get Encounter Order

GET /api/v1/encounters/{orderId}

Poll the status of a previously submitted encounter order.
Authentication: TrainerToken (default)

Path parameters

Name	Type	Required	Description
orderId	string	Yes	Encounter order identifier. · Example: ord_demo_0001

Responses Common errors ↗

Status	Description	Schema
200	Encounter order status	↗ EncounterOrderResponse

200 Response examples

Queued order `queued`


```
{
  "orderId": "ord_demo_0001",
  "status": "QUEUED",
  "submittedAt": "2026-04-01T10:00:00Z"
}
```

Regions

Regions, habitats and the routes inside them. 1 endpoint

List Region Habitats

GET /api/v1/regions/{regionId}/habitats

List habitats for a region, including nested sub-habitats.
Authentication: TrainerToken (default)

Path parameters

Name	Type	Required	Description
regionId	string	Yes	Stable region identifier. · Example: reg_lumen

Responses

Common errors ↗

Status	Description	Schema
200	Region habitats	↗ HabitatListResponse

200 Response examples

Lumen Vale habitats habitats


```
{
  "regionId": "reg_lumen",
  "items": [
    {
      "habitatId": "hab_meadow",
      "name": "Sunlit Meadow",
      "climate": "TEMPERATE",
      "children": [
        {
          "habitatId": "hab_west_meadow",
          "name": "West Meadow",
          "climate": "TEMPERATE"
        }
      ]
    }
  ]
}
```

Items

Item catalogue browsing and catalogue imports. 2 endpoints

List Items

GET /api/v1/items

Browse the item catalogue by category.
Authentication: TrainerToken (default)

Query parameters

Name	Type	Required	Description
category	↗ ItemCategory	—	Filter by item category.
pageNo	integer default: 1 · min: 1	—	Page number, starting at 1. · Example: 1
pageSize	integer default: 20 · min: 1 · max: 100	—	Number of items per page. · Example: 20

Responses Common errors ↗

Status	Description	Schema
200	A page of items	↗ ItemPage

200 Response examples

Medicine and berries items


```
{
  "items": [
    {
      "itemId": "itm_glow_salve",
      "name": "Glow Salve",
      "category": "MEDICINE",
      "cost": 320
    },
    {
      "itemId": "itm_cinder_berry",
      "name": "Cinder Berry",
      "category": "BERRY",
      "cost": 80
    }
  ],
  "pagination": {
    "pageNo": 1,
    "pageSize": 20,
    "total": 2
  }
}
```

Import Item Catalogue

POST /api/v1/items/import

Upload a CSV item catalogue. The import is processed synchronously; a successful import returns no response body.

Authentication: TrainerToken (default)

Request body Required

multipart/form-data Schema: object

Field	Type	Required	Description
file	binary	Yes	

Small CSV catalogue catalogue

```
{
  "file": "item-catalogue.csv"
}
```

Replacement catalogue replacement

```
{
  "file": "item-catalogue-v2.csv",
  "replaceExisting": true
}
```

Responses Common errors ↗

Status	Description	Schema
204	Created (No Content) no body	—

Breeding

1 endpoint

List Breeding Pairs

`GET` `/api/v1/breeding/pairs`

List compatible breeding pairs discovered by the demo service.
Authentication: `TrainerToken` (default)

Responses

Common errors ↗

Status	Description	Schema
200	Breeding pairs	↗ `BreedingPairList`

Data Models

Reusable request and response models are defined once here. Endpoint sections link to these definitions instead of repeating fields. Error payload models are documented with the error conventions.

Primary models

Request and response models developers are most likely to inspect directly.

SpeciesStatsResponse

`object` 6 properties · 0 required

Response wrapper for species stats.

Field	Type	Req.	Constraints	Description
hp	integer	—		Base HP. Example: 45
attack	integer	—		Base attack. Example: 60
defense	integer	—		Base defense. Example: 40
speed	integer	—		Base speed. Example: 65
special	integer	—		Base special attack. Example: 55
total	integer	—		Sum of all base stats. Example: 265

TrainerCreatedResponse

`object` 5 properties · 4 required

Trainer profile returned after registration.

Field	Type	Req.	Constraints	Description
trainerId	string	Yes		Stable trainer identifier. Example: trn_mira
displayName	string	Yes		Public trainer name. Example: Mira Vale
homeRegion	string	Yes		Home region name. Example: Lumen Vale
favoriteType	↗ PokemonType	Yes		
createdAt	date-time	—		Registration timestamp. Example: 2026-02-14T09:30:00Z

TrainerRegistration

`object` 4 properties · 3 required

Fields required to register a trainer.

Field	Type	Req.	Constraints	Description
displayName	string	Yes	min length: 2 · max length: 40	Public trainer name. Example: `Mira Vale`
homeRegion	string	Yes		Name of the trainer's home region. Example: `Lumen Vale`
favoriteType	↗ PokemonType	Yes		
starterSpeciesId	string	—		Optional starter species identifier. Example: `spc_leafling`

Habitat

`object` 4 properties · 3 required

A habitat, possibly containing nested sub-habitats.

Field	Type	Req.	Constraints	Description
habitatId	string	Yes		Stable habitat identifier. Example: `hab_meadow`
name	string	Yes		Habitat name. Example: `Sunlit Meadow`
climate	↗ HabitatClimate	Yes		
children	array<↗ Habitat>	—		Nested sub-habitats.

PartyMember

`object` 4 properties · 2 required

A single Pokémon in a trainer party.

Field	Type	Req.	Constraints	Description
speciesId	string	Yes		Species identifier. Example: `spc_emberling`
nickname	string	—		Optional nickname. Example: `Cinder`
level	integer	Yes	min: 1 · max: 100	Current level. Example: 24
role	↗ PartyRole	—		

SpeciesDetail

`object` 10 properties · 8 required

Compact species record for list views.

Field	Type	Req.	Constraints	Description
speciesId	string	Yes		Stable species identifier.

Field	Type	Req.	Constraints	Description
				Example: `spc_emberling`
name	`string`	Yes		Species name. Example: `Emberling`
types	`array<↗ PokemonType>`	Yes		Elemental types.
habitat	`string`	—		Primary habitat name. Example: `Ember Cavern`
rarity	↗ Rarity	Yes		
description	`string`	Yes		Natural-history description. Example: `A small fox-like species that stores warmth in its tail.`
heightM	`float`	Yes		Height in metres. Example: `0.6`
weightKg	`float`	Yes		Weight in kilograms. Example: `7.4`
abilities	`array<string>`	Yes		Known abilities. Example: `["Kindle","Warm Coat"]`
stats	↗ SpeciesStats	—		

Versioned models

Only differences from the base model are shown; unchanged fields remain documented once.

V2SpeciesStats

differences from previous version Base model: SpeciesStats

Base stats with balance revision metadata.

Field	Change	Detail
balanceRevision	Added	–
updatedAt	Added	–

Unchanged fields are defined by the base model. No differences are hidden.

V2SpeciesStatsResponse

differences from previous version Base model: SpeciesStatsResponse

Response wrapper for species stats with revision metadata.

Field	Change	Detail
balanceRevision	Added	–
updatedAt	Added	–

Unchanged fields are defined by the base model. No differences are hidden.

Response envelopes

Simple wrappers around a result object or status code.

EncounterOrderResponse

response envelope

Response wrapper for an encounter order.

Field	Type	Required
orderId	string	—
status	↗ EncounterStatus	—
submittedAt	date-time	—
result	object	—

HabitatListResponse

response envelope

Habitats belonging to a region.

Field	Type	Required
regionId	string	—
items	array<↗ Habitat>	—

SpeciesResponse

response envelope

Response wrapper for a species detail.

Field	Type	Required
data	↗ SpeciesDetail	—

TrainerPartyResponse

response envelope

Response wrapper for a trainer party.

Field	Type	Required
trainerId	string	—
capacity	integer	—
party	array<↗ PartyMember>	—

TrainerTokenResponse

response envelope

Login response envelope.

Field	Type	Required
token	string	Yes
expiresAt	date-time	Yes
trainer	↗ TrainerSummary	—

Small reusable models

AcceptedOrder

object 3 properties · 2 required

Order accepted for asynchronous processing.

Field	Type	Req.	Constraints	Description
orderId	string	Yes		Order identifier used for polling. Example: ord_demo_0001
status	↗ EncounterStatus	Yes		
estimatedAt	date-time	—		Estimated completion timestamp. Example: 2026-04-01T10:00:05Z

BreedingPair

object 3 properties · 2 required
A compatible breeding pair.

Field	Type	Req.	Constraints	Description
speciesA	string	Yes		First species identifier. Example: spc_emberling
speciesB	string	Yes		Second species identifier. Example: spc_aquafin
compatibility	float	—	min: 0 · max: 1	Compatibility score. Example: 0.72

BreedingPairList

object 1 properties · 0 required
List of breeding pairs.

Field	Type	Req.	Constraints	Description
items	array<↗ BreedingPair>	—		Matching pairs.

EncounterMember

object 2 properties · 2 required
A species observed during a party encounter.

Field	Type	Req.	Constraints	Description
speciesId	string	Yes		Species identifier. Example: spc_aquafin
level	integer	Yes	min: 1 · max: 100	Observed level. Example: 11

EncounterStatus

string
Processing state of an encounter order.

string
enum: QUEUED · PROCESSING · COMPLETE · FAILED

HabitatClimate

string
Broad climate classification for a habitat.

string
enum: TEMPERATE · ARID · TUNDRA · TROPICAL

ItemCategory

string
Functional category for an item.

string
enum: MEDICINE · BALL · BERRY · KEY_ITEM · TM

ItemPage

`object` 2 properties · 2 required
Paginated item list.

Field	Type	Req.	Constraints	Description
items	array<↗ Item>	Yes		Items on this page.
pagination	↗ Pagination	Yes		

Pagination

`object` 3 properties · 3 required
Pagination metadata for a list response.

Field	Type	Req.	Constraints	Description
pageNo	integer	Yes	min: 1	Current page number. Example: 1
pageSize	integer	Yes	min: 1 · max: 100	Number of items per page. Example: 20
total	integer	Yes	min: 0	Total number of matching items. Example: 42

PartyEncounterRequest

`object` 2 properties · 2 required
Record an encounter involving multiple species.

Field	Type	Req.	Constraints	Description
members	array<↗ EncounterMember>	Yes	min items: 2	Observed party members.
location	string	Yes		Free-text location label. Example: Mistwood

PartyRole

`string`
Intended battle role for a party member.

string
enum: ATTACKER · SUPPORT · TANK

PokemonType

`string`
Elemental type used by species and moves.

string
enum: FIRE · WATER · GRASS · ELECTRIC · ROCK · GHOST · FLYING · NORMAL · ICE

Rarity

`string`
Relative rarity of a species.

string
enum: COMMON · UNCOMMON · RARE · LEGENDARY

RegionGeneration

`string`
Generation label for a region.

string
enum: GEN1 · GEN2 · GEN3

RouteEncounterRequest

`object` 3 properties · 3 required
Record a route sweep.

Field	Type	Req.	Constraints	Description
routeId	string	Yes		Route identifier. Example: `rte_west_meadow`
count	integer	Yes	min: 1 · max: 50	Number of encounters to record. Example: 5
location	string	Yes		Free-text location label. Example: `Sunlit Meadow`

SingleEncounterRequest

`object` 3 properties · 3 required
Record one wild encounter.

Field	Type	Req.	Constraints	Description
speciesId	string	Yes		Species identifier. Example: `spc_emberling`
level	integer	Yes	min: 1 · max: 100	Observed level. Example: 12
location	string	Yes		Free-text location label. Example: `Ember Cavern`

SortOrder

`string`
Sort direction for list endpoints.

string
enum: ASC · DESC

SpeciesPage

`object` 2 properties · 2 required
Paginated species list.

Field	Type	Req.	Constraints	Description
items	array<↗ SpeciesSummary>	Yes		Species on this page.
pagination	↗ Pagination	Yes		

SpeciesSummaryList

`object` 1 properties · 0 required
Simple species list.

Field	Type	Req.	Constraints	Description
items	array<↗ SpeciesSummary>	—		Matching species.

TrainerLogin

object 2 properties · 2 required
Trainer credentials used to obtain a bearer token.

Field	Type	Req.	Constraints	Description
username	string	Yes		Trainer username. Example: mira.vale
password	string	Yes	write-only	Trainer password. Example: demo-password

TrainerParty

object 3 properties · 2 required
A trainer's active party.

Field	Type	Req.	Constraints	Description
trainerId	string	Yes		Stable trainer identifier. Example: trn_mira
capacity	integer	—	min: 1 · max: 6	Maximum party size. Example: 6
party	array<↗ PartyMember>	Yes	max items: 6	Active party members.

TrainerPartyUpdate

object 3 properties · 1 required
Replacement party payload.

Field	Type	Req.	Constraints	Description
party	array<↗ PartyMember>	Yes	max items: 6	Replacement party members.
favoriteType	↗ PokemonType	—		
notes	string	—	max length: 200	Optional trainer notes. Example: Keep the party balanced for cave routes.

TrainerSummary

object 3 properties · 2 required
Compact trainer reference used by nested responses.

Field	Type	Req.	Constraints	Description
trainerId	string	Yes		Stable trainer identifier. Example: trn_mira
displayName	string	Yes		Public trainer name. Example: Mira Vale
region	string	—		Home region name. Example: Lumen Vale

TrainerToken

`object` 3 properties · 2 required

Bearer token returned after a successful login.

Field	Type	Req.	Constraints	Description
token	string	Yes		Bearer token value. Example: `pkdx_demo_7f4c2a`
expiresAt	date-time	Yes		Token expiry timestamp. Example: `2026-12-31T23:59:59Z`
trainer	↗ TrainerSummary	—		

TypeEffectiveness

`map<string, float>`

Effectiveness multipliers keyed by attacking type.

Free-form object.

Additional properties: `float`

Helper models

Referenced by other models; grouped here rather than given equal weight to public models.

EncounterOrder

`object` 4 properties · 3 required

Current state of an encounter order.

Field	Type	Req.	Constraints	Description
orderId	string	Yes		Order identifier. Example: `ord_demo_0001`
status	↗ EncounterStatus	Yes		
submittedAt	date-time	Yes		Submission timestamp. Example: `2026-04-01T10:00:00Z`
result	object	—		Optional result payload once processing completes.

↳ **result** nested object

Field	Type	Req.	Constraints	Description
encountered	integer	—		Number of encounters recorded. Example: 3

Item

`object` 5 properties · 4 required
An item in the catalogue.

Field	Type	Req.	Constraints	Description
itemId	string	Yes		Stable item identifier. Example: `itm_glow_salve`
name	string	Yes		Item name. Example: `Glow Salve`
category	↗ ItemCategory	Yes		
cost	integer	Yes	min: 0	Cost in Pocket Coins. Example: `320`
description	string	—		Item description. Example: `Restores a small amount of health.`

Region

`object` 4 properties · 3 required
A PocketDex region.

Field	Type	Req.	Constraints	Description
regionId	string	Yes		Stable region identifier. Example: `reg_lumen`
name	string	Yes		Public region name. Example: `Lumen Vale`
generation	↗ RegionGeneration	Yes		
habitats	array<↗ Habitat>	—		Habitats in the region.

SpeciesStats

`object` 6 properties · 6 required

Base battle statistics for a species.

Field	Type	Req.	Constraints	Description
`hp`	`integer`	Yes		Base HP. Example: `45`
`attack`	`integer`	Yes		Base attack. Example: `60`
`defense`	`integer`	Yes		Base defense. Example: `40`
`speed`	`integer`	Yes		Base speed. Example: `65`
`special`	`integer`	Yes		Base special attack. Example: `55`
`total`	`integer`	Yes		Sum of all base stats. Example: `265`

SpeciesSummary

`object` 5 properties · 4 required

Compact species record for list views.

Field	Type	Req.	Constraints	Description
`speciesId`	`string`	Yes		Stable species identifier. Example: `spc_emberling`
`name`	`string`	Yes		Species name. Example: `Emberling`
`types`	`array<↗ PokemonType>`	Yes		Elemental types.
`habitat`	`string`	—		Primary habitat name. Example: `Ember Cavern`
`rarity`	↗ Rarity	Yes		

Enumerations

Allowed values for enumerated fields and request parameters.

Name	Allowed values	Used in
EncounterStatus	QUEUED · PROCESSING · COMPLETE · FAILED	EncounterStatus
HabitatClimate	TEMPERATE · ARID · TUNDRA · TROPICAL	HabitatClimate
ItemCategory	MEDICINE · BALL · BERRY · KEY_ITEM · TM	ItemCategory
PartyRole	ATTACKER · SUPPORT · TANK	PartyRole
PokemonType	FIRE · WATER · GRASS · ELECTRIC · ROCK · GHOST · FLYING · NORMAL · ICE	PokemonType
Rarity	COMMON · UNCOMMON · RARE · LEGENDARY	Rarity
RegionGeneration	GEN1 · GEN2 · GEN3	RegionGeneration
sortBy	name · rarity · speciesId	Request parameters
SortOrder	ASC · DESC	SortOrder

Specification Notes

The following observations come from inconsistencies or conventions in the source specification.

- Responses 4XX and 5XX are documented as HTTP status-class ranges, not individual status codes.
- The tag 'Breeding' is used by operations but not declared in the top-level tags list; it is documented as its own section here.
- Duplicate operation summaries ('Get Species Stats') are disambiguated with the API version in the endpoint headings.
- Potential specification inconsistency: response 204 is described as "Created (No Content)".